

Java `StringBuilder` and `StringBuffer`

In Java, `String` is immutable, meaning **every modification creates a new object in memory**. To handle frequent string manipulations efficiently, Java provides `StringBuilder` and `StringBuffer`, which are **mutable** alternatives to `String`.

Understanding `StringBuilder`

Why `StringBuilder`?

- It is **mutable**, meaning it modifies the same object instead of creating a new one.
- **Fast** because it is **not synchronized** (no locking mechanisms).
- Ideal for **single-threaded applications**.

Example of `StringBuilder` java

```
public class StringBuilderExample {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder("Hello");  
  
        sb.append(" World"); // Appends " World"  
        sb.insert(5, ",");    // Inserts ", " at index 5  
        sb.replace(6, 11, "Java"); // Replaces "World" with "Java"  
        sb.delete(5, 6);      // Removes the comma  
        sb.reverse();         // Reverses the string  
  
        System.out.println(sb); // Output: "avaJ olleH"  
    }  
}
```

Understanding StringBuffer

Why StringBuffer?

- Similar to `StringBuilder` but **synchronized**, meaning it is **thread-safe**.
- Slightly **slower** due to synchronization overhead.
- Used in **multi-threaded applications**.

Example of StringBuffer

```
public class StringBufferExample {  
    public static void main(String[] args) {  
        StringBuffer sb = new StringBuffer("Hello");  
  
        sb.append(" World");  
        sb.insert(5, ",");  
        sb.replace(6, 11, "Java");  
        sb.delete(5, 6);  
        sb.reverse();  
  
        System.out.println(sb); // Output: "avaJ olleH"  
    }  
}
```

Key Differences Between `StringBuilder` and `StringBuffer`

Feature	<code>StringBuilder</code>	<code>StringBuffer</code>
Thread-Safety	❌ Not thread-safe	✅ Thread-safe
Performance	🚀 Faster (No sync overhead)	🐢 Slower (Due to synchronization)
Best Use Case	Single-threaded operations	Multi-threaded operations
Example	<code>StringBuilder sb = new StringBuilder("Hello");</code>	<code>StringBuffer sb = new StringBuffer("Hello");</code>

Performance Comparison: **StringBuilder** vs **StringBuffer**

```
public class PerformanceTest {
    public static void main(String[] args) {
        long startTime, endTime;

        // Testing StringBuilder Performance
        startTime = System.nanoTime();
        StringBuilder sb = new StringBuilder("Java");
        for (int i = 0; i < 100000; i++) {
            sb.append(" Performance Test");
        }
        endTime = System.nanoTime();
        System.out.println("StringBuilder Time: " + (endTime -
startTime) + " ns");

        // Testing StringBuffer Performance
        startTime = System.nanoTime();
        StringBuffer sbf = new StringBuffer("Java");
        for (int i = 0; i < 100000; i++) {
            sbf.append(" Performance Test");
        }
        endTime = System.nanoTime();
        System.out.println("StringBuffer Time: " + (endTime -
startTime) + " ns");
    }
}
```

Expected Output:

```
StringBuilder Time: 5xxxxxxx ns
StringBuffer Time: 7xxxxxxx ns
```

👉 **StringBuilder** is faster than **StringBuffer** due to the lack of synchronization.

When to Use What?

- ✓ Use **String** if the value **does not change frequently** (e.g., Constants, keys, messages).
 - ✓ Use **StringBuilder** if the value **changes frequently in a single-threaded** environment (e.g., String concatenation, dynamic strings).
 - ✓ Use **StringBuffer** if **multiple threads** modify the string **simultaneously** (e.g., Logging, report generation, shared data).
-

Thread Safety Example (**StringBuffer** vs **StringBuilder**)

Using **StringBuffer** in Multi-Threaded Environment

```
class StringBufferTask extends Thread {
    StringBuffer sb;

    StringBufferTask(StringBuffer sb) {
        this.sb = sb;
    }

    public void run() {
        for (int i = 0; i < 5; i++) {
            sb.append(i).append(" ");
        }
    }

    public static void main(String[] args) throws InterruptedException
    {
        StringBuffer sb = new StringBuffer("Numbers: ");

        Thread t1 = new StringBufferTask(sb);
        Thread t2 = new StringBufferTask(sb);
```

```

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(sb);
    }
}

```

Output Example (Thread-Safe):

Numbers: 0 1 2 3 4 0 1 2 3 4

👉 Ensures thread safety, no data corruption.

Using **StringBuilder** in Multi-Threaded Environment (Not Thread-Safe)

```

class StringBuilderTask extends Thread {
    StringBuilder sb;

    StringBuilderTask(StringBuilder sb) {
        this.sb = sb;
    }

    public void run() {
        for (int i = 0; i < 5; i++) {
            sb.append(i).append(" ");
        }
    }

    public static void main(String[] args) throws InterruptedException
    {
        StringBuilder sb = new StringBuilder("Numbers: ");

        Thread t1 = new StringBuilderTask(sb);
        Thread t2 = new StringBuilderTask(sb);

        t1.start();
        t2.start();
    }
}

```

```

        t1.join();
        t2.join();

        System.out.println(sb);
    }
}

```

Output Example (Not Thread-Safe - Unpredictable Results):

Numbers: 0 0 1 1 2 2 3 3 4 4

👉 Corrupted output because **StringBuilder** is not thread-safe!

String vs StringBuilder vs StringBuffer

Feature	String (Immutable)	StringBuilder (Mutable)	StringBuffer (Mutable & Thread-Safe)
Mutability	❌ Immutable (New object on change)	✅ Mutable (Efficient)	✅ Mutable (Efficient but slower)
Thread Safety	❌ Not thread-safe	❌ Not thread-safe	✅ Thread-safe (Synchronized)
Performance	🐢 Slow (New object every time)	🚀 Fast (No synchronization overhead)	🐢 Slower (Synchronization overhead)
Best for	Fixed values, small operations	Single-threaded string operations	Multi-threaded environments

Key Takeaways

- ✓ Use **StringBuilder** for fast, single-threaded operations.
- ✓ Use **StringBuffer** for thread-safe operations.
- ✓ Avoid using **String** when modifying data frequently to avoid memory wastage.

Would you like an interview-style Q&A on **StringBuilder** and **StringBuffer**?